로컬 모델(Qwen) 설치

PI·coder·writer·scholar는 로컬 모델 **Qwen3.6-35B-A3B**로 생각한다 (analyst·reviewer는 ChatGPT, 6명 모두 예비는 Claude Opus). 모델은 llama.cpp의 `llama-server`로 띄우고, Hermes는 OpenAI 호환 주소(`http://127.0.0.1:<포트>/v1`)로 연결한다.

두 가지 버전이 있다. 내 환경에 맞는 쪽 하나만 따라 한다.

	1. 개인 데스크탑	2. 공용 서버
하드웨어 예	GPU 1개 (RTX 5080 16GB), RAM 32GB	A4000(16GB) 10개, RAM 240GB, 10명
모델 파일	APEX **I-Compact** (17.3GB)	APEX **I-Quality** (22.8GB)
배치	GPU에 안 들어가는 부분은 CPU·RAM이 나눠 돈다	GPU 2개에 전부 올린다. 서버 5개 × 2명
동시 요청	1 (`-np 1`)	4 (`-np 4`, 슬롯당 131,072 토큰)
실행	systemd 사용자 서비스	systemd 시스템 서비스 5개

> Ollama로도 돌릴 수는 있지만, 이 모델(APEX 양자화, 최신 qwen35moe 구조)과 슬롯·KV 캐시 조정 때문에 llama.cpp를 쓴다.

모델은 Hugging Face `mudler/Qwen3.6-35B-A3B-APEX-GGUF`에 있다.

파일	크기 (bytes)	sha256
`Qwen3.6-35B-A3B-APEX-I-Compact.gguf`	17,293,089,472	`50e1122946854f2272b44d466c03d17d410d3f02dcd1c021a1f29f6b384a7126`
`Qwen3.6-35B-A3B-APEX-I-Quality.gguf`	22,819,402,432	`ffb9535b3133942909c3973272d8e65748edafaae5272273baedc7ad78ad2ea6`

1. 개인 데스크탑

1-1. llama.cpp 빌드 (CUDA)

하는 일: 모델을 돌리는 프로그램(`llama-server`)을 GPU용으로 컴파일한다. 10~20분.

CUDA Toolkit과 `cmake`가 있어야 한다 (`nvcc --version`, `cmake --version`).

```
git clone https://github.com/ggml-org/llama.cpp ~/llama.cpp
cd ~/llama.cpp && cmake -B build -DGGML_CUDA=ON -DCMAKE_BUILD_TYPE=Release && cmake --build build -j --target llama-server
```

- 지금 연구실에서 쓰는 버전은 commit `8ca4060b7` (2026-10-28)이다. 새 버전에서 문제가 생기면 `git checkout 8ca4060b7` 후 다시 빌드한다.

1-2. 모델 받기

하는 일: 모델 파일(17.3GB)을 받고, 받은 파일이 원본과 같은지 확인한다.

```
mkdir -p ~/models && cd ~/models
uvx --from huggingface_hub hf download mudler/Qwen3.6-35B-A3B-APEX-GGUF Qwen3.6-35B-A3B-APEX-I-Compact.gguf --l
ocal-dir .
sha256sum Qwen3.6-35B-A3B-APEX-I-Compact.gguf     # 위 표의 값과 같은지 확인
```

다른 PC에 이미 있으면 파일을 그대로 복사해도 된다.

1-3. 실행해 보기

하는 일: 서버를 손으로 한 번 띄워 응답하는지 본다. 자동 실행(1-4) 전에 문제를 먼저 잡기 위해서다.

```
~/llama.cpp/build/bin/llama-server -m ~/models/Qwen3.6-35B-A3B-APEX-I-Compact.gguf \
  --alias qwen3.6-35b-a3b --jinja -c 131072 -np 1 -fa on -lm none -fitt 4096 \
  --host 127.0.0.1 --port 8080
```

옵션	뜻
`--alias qwen3.6-35b-a3b`	Hermes가 부르는 모델 이름. Hermes 설정의 `model.default`와 같아야 한다
`--jinja`	모델의 채팅 템플릿 사용 (도구 호출에 필요)
`-c 131072`	문맥 길이. Hermes `model.context_length`와 맞춘다
`-np 1`	동시 요청 1개 (슬롯 하나가 문맥 전체를 쓴다)
`-fa on`	flash attention
`-fitt 4096`	GPU마다 4GB를 남기고 나머지에 모델을 채운다. 넘치는 부분은 CPU·RAM으로
`--host 127.0.0.1`	이 PC 안에서만 접속 가능

다른 터미널에서 확인: `curl -s http://127.0.0.1:8080/v1/models` → `qwen3.6-35b-a3b`가 보이면 된다. 확인 후 Ctrl+C로 끈다.

1-4. 재부팅해도 자동 실행

하는 일: 서버를 systemd 서비스로 등록해, 컴퓨터를 켜면 저절로 뜨고 죽으면 다시 뜨게 한다.

```
mkdir -p ~/.config/systemd/user
cp ~/Projects/hermes-team/extras/llama/llama-server.service ~/.config/systemd/user/
systemctl --user daemon-reload && systemctl --user enable --now llama-server
sudo loginctl enable-linger $USER      # 로그인하지 않아도 돌게
systemctl --user status llama-server   # active (running)
```

로그: `journalctl --user -u llama-server -f`

1-5. Hermes 연결

아래 "3. Hermes 연결"을 포트 `8080`으로 따라 한다.

2. 공용 서버 (A4000 × 10, GPU 2개씩 묶기)

GPU 2개(32GB)에 I-Quality 모델 전체를 올린 서버를 **5개** 띄우고, 서버 하나를 **2명**이 같이 쓴다. 한 사람의 Hermes 직원들은 동시에 1~3개 요청을 보내므로, 서버마다 슬롯 4개(`-np 4`)를 둔다.

> **슬롯은 두 사람이 같이 쓰는 공용이다.** 사람별로 2개씩 예약되지 않고, 먼저 온 요청부터 빈 슬롯에 들어간다. 한 사람이 4개를 다 쓰는 동안 다른 사람의 요청은 실패하지 않고 잠깐 기다린다. 실제 사용량(한 사람 최대 동시 3개, 실제로 돌아가는 시간 약 3%)으로는 겹치는 일이 드물어서 그대로 쓴다. 기다림이 잦아지면 앞에 프록시를 두고 사람마다 동시 요청을 2개로 제한한다.

서버	GPU	포트	사용자
`llama-qwen@1`	0,1	8081	2명
`llama-qwen@2`	2,3	8082	2명
`llama-qwen@3`	4,5	8083	2명
`llama-qwen@4`	6,7	8084	2명
`llama-qwen@5`	8,9	8085	2명

실험용 GPU는 따로 있다는 전제다. 이 10개는 모델 전용으로 둔다.

2-0. 시작 전 확인 (5분)

하는 일: 빌드와 실행에 필요한 것이 서버에 있는지, 모델용 GPU 10개가 비어 있는지 확인한다.

```
nvidia-smi --query-gpu=index,name,memory.total,memory.used,pci.bus_id --format=csv   # A4000 10개, 사용량 0에 가까운지
nvcc --version && cmake --version && git --version     # CUDA Toolkit 12 이상, cmake 3.18 이상
uv --version                                           # 모델 받기에 쓴다 (없으면: curl -LsSf https://astral.sh/uv/install.sh | sh)
df -hT /scratch ~                                      # 설치 위치(/scratch)가 로컬 디스크(ext4·xfs)이고 여유 40GB 이상인지
sudo -v                                                # 관리자 권한
```

- **conda를 끄고 진행한다.** 프롬프트 앞에 `(base)`가 보이면 `conda deactivate`를 먼저 한다. conda의 컴파일러·라이브러리가 섞이면 빌드가 실패하거나, 서비스로 돌릴 때 라이브러리를 못 찾는다.
- **설치 위치는 로컬 디스크로.** 클러스터 노드는 `/opt`·`/home` 등이 login 서버의 NFS(`login:/opt`처럼 보인다)인 경우가 많다. NFS에는 root 권한이 막혀 설치가 실패하거나, 공용 공간을 차지하게 된다. 이 문서는 노드의 로컬 NVMe `/scratch/llama`에 설치한다 (약 30GB).
- **GPU 번호:** 아래 표의 번호는 `nvidia-smi` 순서(PCI 버스 순서)다. 서비스 설정에 `CUDA_DEVICE_ORDER=PCI_BUS_ID`를 넣어 CUDA도 같은 순서로 세게 한다. 실험용 GPU가 같은 서버에 섞여 있으면, 표의 번호를 모델용 GPU 번호로 바꿔 적는다.

2-1. llama.cpp 빌드 (관리자 한 번)

하는 일: 서비스 전용 계정 `llama`를 만들고, `/scratch/llama/llama.cpp`에 GPU용 `llama-server`를 컴파일한다.

A4000은 Ampere(compute capability 8.6)라서 아키텍처를 지정해 빌드한다.

```
sudo mkdir -p /scratch/llama && sudo useradd -r -m -d /scratch/llama/home llama   # 서비스 전용 계정
sudo mkdir -p /scratch/llama/llama.cpp /scratch/llama/models /etc/llama && sudo chown llama: /scratch/llama/lla
ma.cpp && sudo chown $USER: /scratch/llama/models
sudo -u llama git clone https://github.com/ggml-org/llama.cpp /scratch/llama/llama.cpp
sudo -u llama git -C /scratch/llama/llama.cpp checkout 8ca4060b7       # 연구실에서 검증한 버전으로 고정
cd /scratch/llama/llama.cpp && sudo -u llama cmake -B build -DGGML_CUDA=ON -DCMAKE_CUDA_ARCHITECTURES=86 -DCMAK
E_BUILD_TYPE=Release \
  -DCMAKE_CUDA_COMPILER=$(command -v nvcc) && sudo -u llama cmake --build build -j --target llama-server
```

- `-DCMAKE_CUDA_COMPILER`는 `sudo`가 PATH를 바꿔 `llama` 계정에서 `nvcc`가 안 보이는 문제를 막는다. 내 계정에서 찾은 `nvcc` 경로를 그대로 넘긴다 (2-0에서 `nvcc --version`이 나와야 한다).

2-2. 모델 받기 (하나만 받아 5개 서버가 같이 쓴다)

하는 일: 모델 파일(22.8GB)을 `/scratch/llama/models`에 한 번만 받고 원본과 같은지 확인한다.

```
cd /scratch/llama/models
uvx --from huggingface_hub hf download mudler/Qwen3.6-35B-A3B-APEX-GGUF Qwen3.6-35B-A3B-APEX-I-Quality.gguf --l
ocal-dir .
chmod a+r Qwen3.6-35B-A3B-APEX-I-Quality.gguf            # 서비스 계정 llama가 읽을 수 있게
sha256sum Qwen3.6-35B-A3B-APEX-I-Quality.gguf          # 위 표의 값과 같은지 확인
```

2-3. 서버 하나 먼저 띄워 보기

하는 일: GPU 0,1로 서버 하나를 손으로 띄워, 모델이 GPU 2개에 다 들어가는지 확인한다. 여기서 통과한 옵션을 2-4에서 그대로 쓴다.

```
sudo -u llama env CUDA_DEVICE_ORDER=PCI_BUS_ID CUDA_VISIBLE_DEVICES=0,1 /scratch/llama/llama.cpp/build/bin/llam
a-server \
  -m /scratch/llama/models/Qwen3.6-35B-A3B-APEX-I-Quality.gguf --alias qwen3.6-35b-a3b --jinja \
  -ngl all -c 524288 -np 4 -ctk q8_0 -ctv q8_0 -fa on -lm none --host 127.0.0.1 --port 8081
```

옵션	뜻
`CUDA_VISIBLE_DEVICES=0,1`	이 서버에는 GPU 0·1만 보인다. 다른 서버의 GPU를 건드리지 않는다
`-ngl all`	모든 층을 GPU에 올린다. 보이는 GPU 2개에 층과 KV 캐시를 빈 메모리 비율대로 자동으로 나눈다
`-c 524288 -np 4`	전체 문맥을 슬롯 4개가 나눠 쓴다 → 슬롯당 131,072 토큰 (모델 학습 길이 262,144 이내)
`-ctk q8_0 -ctv q8_0`	KV 캐시를 8비트로 저장해 메모리를 절반으로

메모리 확인 (꼭): 계산상 모델 22.8GB + KV 캐시 약 5.6GB + 작업 버퍼 1~2GB로 32GB에 빠듯하게 들어간다.

- 다른 터미널에서 `nvidia-smi`로 **0번·1번 GPU만** 사용량이 올라갔는지 보고, `curl -s http://127.0.0.1:8081/v1/models`로 응답을 확인한다.
- 정확히 반반은 아니다. 출력층과 작업 버퍼가 첫 GPU에 더 실려 보통 0번이 1~2GB 더 쓴다. 어떻게 나뉘었는지는 시작 로그의 `CUDA0`·`CUDA1` 줄에 나온다.
- 한쪽 GPU만 넘쳐 `out of memory`가 나면 `-ts 45,55`(0번 45%, 1번 55%)를 붙여 1번에 더 싣는다.

- 그래도 모자라면 `-c 393216`(슬롯당 98,304)으로 줄이거나 `-np 3 -c 393216`으로 바꾼다. 바꾼 값은 서비스 파일과 Hermes `context_length`에도 똑같이 적는다.

2-4. 5개를 서비스로 등록

하는 일: 서비스 파일 하나로 서버 5개를 등록한다. 서버마다 쓸 GPU와 포트는 `/etc/llama/qwen-N.env`에 적는다. 2-3에서 띄운 서버는 먼저 Ctrl+C로 끈다.

```
sudo cp ~/Projects/hermes-team/extras/llama/llama-qwen@.service /etc/systemd/system/
for n in 1 2 3 4 5; do
  printf 'CUDA_DEVICE_ORDER=PCI_BUS_ID\nCUDA_VISIBLE_DEVICES=%s,%s\nPORT=%s\n' $((2*n-2)) $((2*n-1)) $((8080+n)) | sudo tee /etc/llama/qwen-$n.env >/dev/null
done
sudo systemctl daemon-reload
sudo systemctl enable --now llama-qwen@{1..5}
systemctl status 'llama-qwen@*' --no-pager | grep -E "llama-qwen@|Active"
```

- 로그: `journalctl -u llama-qwen@1 -f`
- 설정 파일 형식은 `extras/llama/qwen.env.example`에 있다.
- 서버는 `127.0.0.1`에만 열려 있으므로, 사용자들은 **같은 서버에 로그인해서** 각자 Hermes를 돌린다. 다른 컴퓨터에서 접속하게 하려면 `--host 0.0.0.0`과 `--api-key`를 같이 쓰고, 방화벽으로 연구실 네트워크만 허용한다.

2-5. 사용자별 Hermes 연결

하는 일: 각 사용자가 자기 Hermes를 배정된 서버(포트)에 연결한다.

사람마다 배정된 포트로 "3. Hermes 연결"을 따라 한다 (예: 1·2번 사용자는 8081, 3·4번 사용자는 8082).
`context_length`는 **슬롯당 길이(131072)**를 쓴다.

2-6. 운영 (끄기·다시 켜기·업데이트)

```
sudo systemctl restart llama-qwen@2                 # 한 대만 다시 켜기 (그 서버를 쓰는 2명만 잠깐 끊긴다)
sudo systemctl stop 'llama-qwen@*'                  # 전부 끄기 (GPU 점검 등)
sudo systemctl start llama-qwen@{1..5}
nvidia-smi                                          # 서버마다 GPU 2개를 쓰는지
```

- **llama.cpp 업데이트:** 2-1에서 특정 버전으로 고정했으므로 `pull`만 하면 실패한다. 최신 코드로 옮긴 뒤 받는다: `sudo -u llama git -C /scratch/llama/llama.cpp checkout master && sudo -u llama git -C /scratch/llama/llama.cpp pull` → 2-1의 마지막 명령(cmake 두 줄) → `sudo systemctl restart 'llama-qwen@*'`. 새 버전이 이상하면 `sudo -u llama git -C /scratch/llama/llama.cpp checkout 8ca4060b7`로 되돌려 다시 빌드한다.
- **옵션 바꾸기:** `/etc/systemd/system/llama-qwen@.service`의 `ExecStart` 줄을 고친 뒤 `sudo systemctl daemon-reload && sudo systemctl restart 'llama-qwen@*'`.
- 서버를 다시 켜는 동안(1~2분) 그 서버를 쓰는 Hermes 직원은 요청이 실패하면 예비 모델(Claude)로 넘어간다. 바쁜 시간에는 재시작을 피한다.

2-7. 클러스터(gnode1)에서 여러 명이 쓸 때

하는 일: 각자 SSH로 gnode1에 들어가 자기 계정에 Hermes를 설치하고 돌린다. 그때 지킬 것을 정리한다.

폴더는 이렇게 둔다

무엇	위치	이유
Hermes (`~/.hermes`)	홈 그대로 (login의 NFS)	아래 규칙만 지키면 그대로 써도 된다
연구 폴더 (`~/Projects`)	홈 그대로	Slurm 작업이 다른 노드에서 돌아도, login 서버에서도 같은 경로가 보인다
모델 (llama.cpp, gguf)	`/scratch/llama`	2-1·2-2
큰 데이터·체크포인트 (필요하면)	`/scratch/<계정>/`	수십 GB 이상일 때. 단, gnode1에서만 보인다

규칙: 내 Hermes는 gnode1에서만 켠다. 홈은 login 서버와 다른 노드에도 보인다. 같은 계정의 `hermes`나 게이트웨이를 login 서버 등에서 동시에 켜면, 두 컴퓨터가 같은 대화 DB(`state.db`, `kanban.db`)를 열어 꼬일 수 있다. gnode1 한 대에서만 쓰면 문제없다.

설치 후 DB 모드 확인 (`wal`이든 `delete`든 정상이다. NFS에서 WAL이 안 되면 Hermes가 스스로 `delete`로 바꾼다):

```
python3 -c "import sqlite3,os;[print(f, sqlite3.connect(os.path.expanduser('~/.hermes/'+f)).execute('pragma journal_mode').fetchone()[0]) for f in ('state.db','kanban.db')]"
```

`database is locked` 오류가 자주 보이면 `~/.hermes/config.yaml`에 아래를 넣고 게이트웨이를 다시 켠다:

```
database:
  journal_mode: delete
```

Discord 연결 확인 (서버에서 한 번) 게이트웨이는 서버에서 Discord로 나가는 연결만 쓴다 (들어오는 포트를 열 필요 없음).

```
curl -sS --max-time 10 -o /dev/null -w '%{http_code}\n' https://discord.com/api/v10/gateway   # 200이면 OK
```

- 아무것도 안 나오고 멈추면 `-v`를 붙여 어디서 멈추는지 본다. gnode1에서는 discord.com 주소 10개 중 2개가 응답하지 않지만, 다른 주소로 다시 시도하므로 그대로 써도 된다 (2026-11-03 확인).
- 프록시가 있는 망이면 `~/.hermes/.env`에 `HTTPS_PROXY`를 넣는다 (gnode1은 프록시 없음).

SSH를 끊어도 계속 돌게 게이트웨이를 SSH 창에서 그냥 실행하면 접속을 끊을 때 같이 꺼진다. 서비스로 등록하고 linger를 켜면 SSH를 끊어도, 재부팅해도 계속 돈다 (INSTALL 4-9).

```
sudo loginctl enable-linger $USER      # 계정마다 한 번
hermes gateway install && hermes gateway start
hermes gateway status        # running
```

각자 연결할 포트는 2-5의 표대로 한다 (1·2번 → 8081, 3·4번 → 8082 …). 같은 서버에 로그인한 사람은 누구나 모든 포트에 닿을 수 있으므로, 자기 포트를 정확히 적는 것은 약속으로 지킨다.

3. Hermes 연결 (두 버전 공통)

하는 일: Hermes 직원 4명(PI·coder·writer·scholar)이 로컬 모델로 생각하게 하고, 모델이 꺼지면 Claude로 넘어가게 한다.

`<포트>`는 데스크탑이면 `8080`, 서버면 배정된 포트다.

① **PI 설정** — `~/.hermes/config.yaml`에 아래를 넣는다:

```
model:
  default: "qwen3.6-35b-a3b"
  provider: "custom:local-qwen"
  context_length: 131072
providers:
  local-qwen:
    base_url: "http://127.0.0.1:<포트>/v1"
    default_model: "qwen3.6-35b-a3b"
fallback_providers:          # 로컬 모델이 꺼져 있거나 실패하면 Claude로
  - provider: claude-subscription-directsdk-experimental
    model: claude-opus-5-5[1m]
    base_url: process://claude-subscription-directsdk-experimental
    api_mode: chat_completions
auxiliary:
  compression:              # 대화 압축 요약은 로컬 모델 대신 GPT로
    provider: openai-codex
    model: gpt-6.1-terra
```

② **다른 직원** — coder·writer·scholar의 `~/.hermes/profiles/<직원>/config.yaml`에도 `model`·`providers`·`fallback_providers`를 같은 값으로 넣는다. analyst·reviewer는 ChatGPT 그대로 둔다.

③ **게이트웨이 재시작:** `hermes gateway restart`

④ **시험:** `hermes chat -q "OK라고만 답해" --oneshot`, `hermes -p coder chat -q "OK라고만 답해" --oneshot`

문제가 생기면:

- `connection refused` → 모델 서버가 꺼져 있다. 데스크탑은 `systemctl --user status llama-server`, 서버는 `systemctl status llama-qwen@<번호>`로 확인한다.
- 답이 느리거나 중간에 끊긴다 → 슬롯이 모자란 것이다. 서버 로그에서 `slot` 대기 메시지를 확인한다.
- 모델 이름 오류 → `--alias`와 Hermes `model.default`가 같은지 확인한다.